PAALHU: project status and pending work

Restaurant website with bookings and orders · 28 October 2026 · Based on the "From Screens to Endpoints" worksheet

1. Summary

Part	Status	Notes
Public screens (home, menu, set menu, spice guide, opening hours, location, about us)	Done	Static HTML, no server data needed
Chat widget on the website	Built, switched off	No AI model connected yet (`CHAT_ENABLED = false`)
Booking logic (model)	Mostly done	In `reservas-core.mjs`, stored in a JSON file, not SQLite
Validators	Written	All in `src/validators.js` (committed). No tests yet; not all of them are used yet
Express	Installed, not used	Express 5.3.0 is in `package.json`, but the server still uses plain Node `http`
Express structure (`app.js`, `routes/`, `middleware/`)	Not started	See section 2
Endpoints	2 of 18	Only `GET /api/chat/status` and `POST /api/chat`; 16 left
Orders (model)	Not started	Only a script that prices an order; nothing saves orders
Database (SQLite)	Not started	Bookings are in a JSON file
Staff screens (login, bookings, orders)	Not started	
Staff login	Not started	
Pagination and filters	Not started	The `page` and `limit` validators already exist
Tests	Not started	
`README.md` and `.env`	Not started	

> **Agreed decisions:** customers can book and order through the chat, by phone (+34 627 41 09 35) or by email (owners@paalhu.es). The chat can only do what staff can do by hand. Staff register phone and email bookings and orders through a staff web page with a login. Data will be stored in SQLite

(built into Node 24, nothing to install). The server follows the course structure (`app.js` , `server.js` , `middleware/` , `routes/` , `controllers/`). Validators only say what is wrong and in which field; the controllers choose the HTTP status code (400, 401, 404…).

2. Project structure

The whole repository, as it is today and as it has to end up. ✅ exists · ⚠️ exists, needs changes · ❌ missing

```
PAALHU/
├── package.json              ✅ Name, "npm start", dependencies (Express 5.3.0)
├── package-lock.json         ✅ Exact installed versions (automatic, don't edit)
├── .gitignore                ✅ node_modules, customer data, booking history
├── README.md                 ❌ How to install, run and use the project
├── .env                      ❌ Settings: port, session secret (git-ignored)
│
├── pages/                    ✅ Public website (static HTML)
│   ├── index.html               Home page, with the chat widget
│   ├── menu.html                Menu and prices
│   ├── set-menu.html            Set menu (Mon–Fri lunch)
│   ├── spice_guide.html         Spice levels 1–5
│   ├── opening_hours.html       Opening hours
│   ├── location.html            Address and how to get there
│   ├── about_us.html            The family / the restaurant
│   ├── booking.html             How to book (chat, phone, email), with the chat widget
│   └── admin/                ❌ Staff pages: login, bookings, orders
├── css/                      ✅ styles.css (website), chat.css (chat widget)
├── js/chat.js                ✅ Chat widget: calls /api/chat/status and /api/chat
├── img/                      ✅ Photos (dining room, family)
├── docs/                     ✅ This document
│
├── src/                         ── the server ──
│   ├── server.js             ⚠️ Today does everything with Node http.
│   │                            Must only import app and call app.listen(PORT)
│   ├── app.js                ❌ Creates the Express app: middleware, routes, static
│   │                            files, 404, error handler. Exports it
│   ├── middleware/           ❌
│   │   ├── logger.js            Logs each request (method, URL, status, time)
│   │   ├── auth.js              Staff login check → 401
│   │   └── errorHandler.js      Errors → HTTP codes (400/401/404/409/500)
│   ├── routes/               ❌ URL → controller function (express.Router)
│   │   ├── chat.js              The 2 chat endpoints (today inside server.js)
│   │   └── auth.js, bookings.js, waitingList.js, orders.js, dishes.js
│   ├── controllers/          ⚠️ Request logic: (req, res) functions
│   │   ├── chatController.js ⚠️ Chat assistant; needs (req, res) handlers
│   │   ├── routes.js         ⚠️ Empty, wrong folder → delete
│   │   └── auth, booking, waitingList, order, dishController.js  ❌
│   ├── models/               ❌ Data access and operations
│   │   ├── booking.js           Wraps reservas-core.mjs (logic already exists)
│   │   └── order.js, dish.js, staff.js
│   ├── db/                   ❌ SQLite schema (migration) and connection
│   └── validators.js         ✅ Input validation (all validators written)
│
├── test/                     ❌ node --test: validators, Order model, endpoints
│
└── .claude/skills/           ✅ Tools for Claude and staff (not part of the website)
    ├── paalhu-assistant/
    │   ├── SKILL.md             Instructions: answer customers, price orders
    │   ├── reference.md         Menu, prices, hours (the chat reads it too)
```

```
│   └── order_total_validator.mjs  Prices an order from the menu
└── paalhu-reservas/
    ├── SKILL.md              Instructions for managing bookings
    ├── config.json           Tables, shifts, policy (12 guests, 24 h)
    ├── reservas-core.mjs     Booking engine (availability, tables, waiting list)
    ├── reservas.mjs          Command-line tool for staff bookings
    └── evals/evals.json      Tests for the skill
```

Course structure compared with ours

Course file	Function	Status	What we need
`package.json`	Dependencies and scripts	Done	Express listed; `npm start` runs `src/server.js`
`src/app.js`	Creates and configures the Express app and exports it (no `listen`)	Missing	Move routes, static files, redirect and private-file blocking here from `server.js`
`src/server.js`	Imports `app` and starts it with `app.listen`	Change	Keep only `process.env.TZ`, `PORT` and `app.listen`
`src/middleware/logger.js`	Logs every request	Missing	Create the folder and the logger; later `auth.js` and `errorHandler.js`
`src/routes/*.js`	Only maps URLs to controller functions	Missing	Start with `routes/chat.js`; delete the empty `controllers/routes.js`
`src/controllers/*Controller.js`	`(req, res)` functions: read the request, call the logic, send the answer	Change	`chatController.answer(body)` returns `[status, body]`; add `status(req, res)` and `send(req, res)`
Our extras	`validators.js`, `models/`, `db/`, `docs/`, `test/`	Partial	`validators.js` and `docs/` exist; the others are missing

Endpoints we already have

Endpoint	Function	Where in the code
`GET /api/chat/status`	Tells the website whether the chat is on: `{ enabled: true | false }`. The widget in `js/chat.js` only shows itself when it is `true`. Today it is `false`.	`src/server.js` line 60, reading `CHAT_ENABLED` from `chatController.js`

`POST /api/chat`	Receives `{ session, message }`, returns `{ session, reply }`. Answers: 200 reply · 400 empty message or bad JSON · 429 previous message still running or conversation too long · 502 model failed · 503 chat switched off (always, today).	`chat()` in `src/server.js` (lines 47–53) reads the body and calls `answer()` in `chatController.js`

The same server also redirects `/` to `/pages/index.html` and serves the website's files, blocking `src/`, dot-folders (`.claude/`), `node_modules/` and `package*.json`.

3. What is already done

Screens

- All public pages in `pages/`, listing chat, phone and email for bookings and orders.
- Chat widget (`js/chat.js`) on the home and booking pages.

Booking model (`reservas-core.mjs`)

- Check availability, with alternatives when a shift is full.
- Create, change and cancel bookings; automatic table assignment, including joining adjacent tables.
- Waiting list, shift occupancy, booking sheet, search.
- Permanent history of every change (`reservas-historial.jsonl`).
- Used by both the chat and the staff command-line tool (`reservas.mjs`).

Entities

Entity	Status	Where
Booking, WaitingListEntry	Done	`reservas-core.mjs` → `reservas.json`. Channel stored only as `chat` / `equipo`
HistoryEvent	Done	`reservas-historial.jsonl`
Table, Shift	Done	`config.json`
Dish	Partial	Only as text in `reference.md` and the menu HTML, no ids
Order, OrderItem	Missing	
Staff	Missing	

Validators (`src/validators.js`)

Each validator returns the clean value or throws a `ValidationError` with a readable message and the name of the wrong field. They contain no HTTP status codes.

Area	Validators

Bookings	`date`, `time`, `guests`, `name`, `phone`, `notes`, `table`, `reference`
Orders	`dish`, `drink`, `quantity`, `price`, `spiceLevel`, `items`, `type`, `address`, `orderTime`, `orderStatus`, `statusChange`
Shared	`channel`, `login`, `page`, `limit`

- The chat uses `date`, `guests`, `name` and `phone`; the order-pricing script uses `dish`, `price` and `quantity`. The rest are ready for the new models and controllers.
- The booking model also checks: time inside a shift, 12-guest maximum, adjacent tables, whether the booking fits.

4. What we need to do, in order

Step 1 · Express and project structure

- `src/app.js`: create the Express app and **export** it (no `listen`). Move into it:
  - `express.json()` with a 16 KB limit (as today's `readBody`).
  - `express.static` for `pages/`, `css/`, `js/`, `img/`.
  - The redirect `/` → `/pages/index.html`.
  - Blocking `src/`, `.claude/`, `node_modules/` and `package*.json` (today's `PRIVATE` rule).
- `src/server.js`: only `process.env.TZ = 'Europe/Madrid'`, `PORT`, `import app` and `app.listen`.
- `src/routes/chat.js` with `express.Router()`; `chatController` exports `status(req, res)` and `send(req, res)`, built on today's `answer()`.
- `src/middleware/logger.js`: method, URL, status and time of each request.
- `src/middleware/errorHandler.js`: `ValidationError` → 400, no login → 401, not found → 404, no room or status not allowed → 409, anything else → 500.
- Delete the empty `src/controllers/routes.js`. Create `src/models/`.
- Add `.env` (git-ignored) and `README.md`.

Step 2 · SQLite database

- Schema (migration) with tables: bookings, waiting_list, orders, order_items, dishes, staff, history.
- Move the existing data from `reservas.json` and the history file into the database.
- Rewrite the storage part of `reservas-core.mjs` (the booking logic stays the same).
- Store the channel as `chat`, `phone` or `email`.
- Add the database file to `.gitignore`: it holds customers' personal data.

Step 3 · Models

- `models/` for Booking and WaitingListEntry (wrapping the booking engine).
- New Order model: create, list, get one, change status. The total is calculated on the server from dish prices.
- Dish model: menu dishes with ids and prices, read-only.
- Staff model: username and hashed password.

Step 4 · Use the validators everywhere

- Chat: also check `time`, `notes` and `reference`.
- Staff bookings through `reservas.mjs`: use the same validators.
- New controllers: validate every request body and query with `validators.js` before calling a model.

Step 5 · Controllers and routes (16 endpoints)

#	Controller	Endpoint	Access	Function
1	`authController`	`POST /api/auth/login`	Public	Staff log in
2		`POST /api/auth/logout`	Staff	Staff log out
3	`bookingController`	`GET /api/shifts`	Staff	Shifts of a day
4		`GET /api/availability`	Staff	Is there a table for N guests at a date and time
5		`GET /api/bookings`	Staff	List, with filters and pagination
6		`GET /api/bookings/:id`	Staff	One booking
7		`POST /api/bookings`	Staff	Create (phone or email)
8		`PATCH /api/bookings/:id`	Staff	Change or cancel
9	`waitingListController`	`GET /api/waiting-list`	Staff	List, by date and status
10		`POST /api/waiting-list`	Staff	Add an entry
11		`PATCH /api/waiting-list/:id`	Staff	Update (called, turned into a booking…)
12	`orderController`	`GET /api/orders`	Staff	List, with filters and pagination
13		`GET /api/orders/:id`	Staff	One order
14		`POST /api/orders`	Staff	Create an order
15		`PATCH /api/orders/:id`	Staff	Change status
16	`dishController`	`GET /api/dishes`	Public	Menu with ids and prices
—	`chatController`	`GET /api/chat/status`, `POST /api/chat`	Public	Already exist

Cancelling uses `PATCH` with `status: "cancelled"`, not `DELETE`: bookings and orders are never deleted.

Step 6 · Staff login

- Hashed passwords and a session or token.
- Every staff endpoint answers 401 without a valid login.

Step 7 · Staff pages

- `/admin/login`: login form.
- `/admin/bookings`: day's shifts and occupancy, bookings, waiting list; forms to create, change and cancel.
- `/admin/orders`: list of orders, form to create an order, buttons to change status.

Step 8 · Pagination and filters

- `GET /api/bookings`: `page`, `limit` (20, max 100), `date` / `from` / `to`, `status`, `search`, `channel`, `sort`.
- `GET /api/orders`: `page`, `limit`, `date`, `status`, `channel`, `type`, `sort`.
- `GET /api/waiting-list`: `date`, `status`.
- Paginated answers return `{ data, page, limit, total }`.

Step 9 · Chat orders

- Add a `create_order` tool to the chat that uses the Order model.
- Remove the current rule that tells customers to call to confirm their order.
- Connect an AI model so the chat is switched on.

Step 10 · Tests

- Validators and the Order model, then the endpoints, with Node's built-in `node --test`.

5. Details to take into account

Server and Express

Point	Detail
Order inside `app.js`	logger → `express.json()` → static files → routes → 404 → error handler. The error handler goes last and has 4 arguments `(err, req, res, next)`.
Express 5 and async	Errors thrown in `async` handlers reach the error handler on their own: no `try/catch` in every route.
Madrid time	`process.env.TZ = 'Europe/Madrid'` must be the first line of `server.js`, before anything reads dates (shifts, "tonight", the `date` validator).
Keep the chat working	`js/chat.js` relies on `GET /api/chat/status` returning `{ enabled }` and on the 503 while the chat is off. Same URLs, same answers.
Private files	Never serve `src/`, `.claude/` (customer data lives there), `node_modules/`, `package*.json` or `.env`. Serve only `pages/`, `css/`, `js/`, `img/`.

`.env`	Git-ignored. Holds `PORT` and, later, the session secret. Read it with `node --env-file=.env` (built into Node).
Thin layers	Routes only map URLs; controllers read the request and answer; models own the data and the rules. No HTTP in models or validators.

Errors and status codes

Point	Detail
Who decides the code	Validators throw `ValidationError` (message + field); models throw their own errors. Only the error handler turns them into 400 / 401 / 404 / 409 / 500.
Error answer format	Always `{ error, field? }`. On 500, never send internal details (stack, file paths); log them on the server.
Success codes	200 for reads and changes, 201 for `POST` that creates something.
Don't leak data	"Not found" must be the same whether the booking doesn't exist or the phone doesn't match (the chat already does this).

Data and privacy

Point	Detail
Customer data off GitHub	The database file goes in `.gitignore`, like `reservas.json` and `*-historial.jsonl` today.
What customers see	Never table numbers, nor other customers' names or phones.
Nothing is deleted	Cancelling = `PATCH status: "cancelled"`. Every change is written to the history.
Channel	`chat`, `phone` or `email` (today `chat` / `equipo`). Staff send `phone` or `email`; only the server sets `chat`.
Order total	Calculated on the server from the dish prices in the database; never trust a total or a price sent by the client. Work in cents to avoid rounding errors.
Spanish field names	The booking engine uses `fecha`, `hora`, `personas`, `nombre`, `telefono`, `notas`, `estado`. The API uses English; the model maps them, as `runTool` in `chatController.js` does today.
One booking engine	The chat, the staff CLI (`reservas.mjs`) and the new endpoints must all use the same engine, so the rules are the same everywhere.

Business rules

Point	Detail

Group size	Maximum 12 guests (`config.json` → `grupo_maximo`). Bigger groups write to owners@paalhu.es.
Cancellation	Free with at least 24 h notice. No deposit.
Times	Bookings: arrival time inside a shift (`config.json` → `turnos`); the table is held for the whole shift. Orders: inside opening hours.
Set menu	Monday to Friday, lunch only.
Spice level	1 to 5 for every dish in an order.
Allergens	Not published per dish. Never say a dish is safe or free of an allergen; note allergies on the booking and refer to owners@paalhu.es.
Hours in several places	Opening hours: `reference.md` , the website and `validators.js` (`orderTime`). Shifts: `config.json` . Change them together, or move them to one place.
Order status	received → confirmed → ready → delivered, or cancelled before delivered. Use `statusChange` .

Login and staff

Point	Detail
Passwords	Hashed with `crypto.scrypt` and a salt (built into Node). Never stored or logged in plain text.
Protected routes	Every staff endpoint and every `/admin/*` page goes through `middleware/auth.js` → 401 without a valid login.
Login answer	Same message for wrong username or wrong password (“Wrong username or password”), and a limit on failed attempts.
First staff user	Needs a way to be created (a small script or the migration), since there's no sign-up page.

Quality

Point	Detail
Tests	`node --test` : validators first, then the Order model, then the endpoints. Add `"test"` to the scripts in `package.json` .
`README.md`	Install (`npm install`), run (`npm start`), an example `.env` , and the list of endpoints.
Language	Code, comments and API in English; the booking engine keeps its Spanish names. Customer messages in the customer's language.
Commits	Small commits with a message saying what changed (not just “commit”).

6. Open points to decide

Point	Detail
Maximum guests	`validators.js` accepts 1–14, but the restaurant's maximum is 12. Change the validator to 12.
Same-day bookings	The `date` validator rejects today, so “a table for tonight” fails. The worksheet says only “not in the past”. Decide which rule applies.
Hours in several places	See section 5. Decide whether to move them all into `config.json`.
File rename	`order_total.mjs` was renamed to `order_total_validator.mjs`, but `SKILL.md`, `reference.md` and the script's own usage text still use the old name.
Uncommitted changes	`package.json` and `package-lock.json` (Express) and the empty `src/controllers/routes.js`.
Customer self-service	Customers manage bookings only through the chat. If the course requires it, add public endpoints that check reference + phone.